Hasító tábla: olyan adatszerkezet, amely egy hash függvény segítségével állapítja meg, hogy melyik kulcshoz milyen érték tartozik - így implementál egy asszociatív tömböt. Minden művelet (beillesztés, törlés, keresés) konstans idő alatt fut le, így nagy tábláknál is hatékony a karbantartás és az elérés.
Ha két kulcs hash értéke azonos, nem lehet őket egy helyen tárolni. Erre vannak különböző tárolási technikák.
- Láncolt listás hasító technika: a hash tábla egyes indexein nem egy elemet tárolunk, hanem azon kulcsok és értékek láncolt listáját, amelyeket a hash függvény oda képzett le. Ekkor a táblában való kereséskor a hash indexén levő listát kell bejárnunk a keresett adat kinyeréséhez. Beszúráskor a lista végéhez hozzáfűzzük az új értéket. Törléskor a megfelelő listaelemet távolítjuk el a láncolt listából. Legrosszabb esetben az összes beszúrt elem ugyanarra az indexre kerül, ekkor minden művelet O(n) lépést igényel.
- Nyílt címzés: a láncolással ellentétben egy indexen kizárólag egy értéket tárolunk (a táblában tárolt elemek száma nem haladhatja meg az indexek (egyedi hashek) számát). Beszúráskor a megadott módszerrel a kapott hashtől indulva megkeressük az első szabad indexet, ahova az elem bekerül. Kereséskor ugyanígy járjuk be az egyes indexeket, ekkor O(n) lépésben megállapíthatjuk, hogy a megadott kulcs szerepel-e a táblában. Törléskor az elem indexét nem üresnek, hanem töröltnek jelöljük, így azok a kereső algoritmusok, amelyek ellenőrzik ezt az indexet se adnak hibás eredményt, viszont egy olyan táblában, ahol gyakoriak a beszúrások és törlések a keresés teljesítménye gyorsabban romlik. Egy kulcs helyét az úgynevezett próbálás során keressük meg. Ennek több, ismert módszere létezik:
Lineáris próbálás esetén két kipróbált hely között a távolság konstans méretű, általában 1. Minden egyes sikertelen próba után a megadott intervallummal lép a függvény bal- vagy jobb irányban tovább a tömbön.
Kvadratikus próbálás során a két kipróbált hely között a távolságot egy másodfokú polinom adja,
kettős hashelés során pedig a lépésközöket egy másik hash függvény számolja ki. Ez csökkentve a csoportulások kialakulásának lehetőségét. Fontos, hogy úgy válasszuk meg a másodlagos függvényt, hogy az semmilyen kulcsra ne adjon 0 értéket, hiszen ekkor a lépésköz minden i-re 0 lenne.
A hash táblák teljesítménye arányosan csökken a telítettségi tényező növekedésével, amit fix méretű táblában nem tudunk csökkenteni anélkül, hogy elemeket távolítanánk el belőle, így az egyetlen fennmaradó lehetőség a tábla méretének növelése. Ekkor az elemek újrahashelése szükséges, mivel azok hashe eltérhet az átméretezett tábla függvénye alapján.

Bináris fák: olyan fa adatstruktúrák, ahol minden csomópontnak legfeljebb két leszármazottja lehet. Bináris keresőfákat és bináris kupacokat szoktak belőlük kialakítani. A rajtuk való kereséseknek két jellemző algoritmusa van:
- *Mélységi keresés*: az algoritmus minden lépésnél a legbaloldalibb, még nem bejárt csomópontot választja ki. Ha már nincs több lépés, akkor visszalép, és az utolsó elágazási pontban a következő, még nem bejárt alternatívát választja ki. A kiindulás történhet a gyökérpontból, vagy egy még nem bejárt csomópontból. Találatra leáll.
- *Széltében keresés*: az algoritmus mindig az adott kiindulási csúcsból, vagy az adott rétegből egy lépésben elérhető csúcsokat járja be. A gyökérpontot kiindulási pontnak tekintve a bináris fát olyan módon járja be, hogy mindig a gyökérhez legközelebb eső, még be nem járt csomópontot keresi fel.

B-fák: speciális fa adatstruktúrák, az adatokat rendezett formában tárolják. Egy csomópont (node) egy előre meghat. tartományban változó mennyiségű leszármazottat tartalmazhat. Megengedi a keresést, beszúrást és törlést legrosszabb esetben is logaritmikus időben. Beillesztés és törlés esetén a csomópontok száma változik, és hogy a leszármazott csomópontok száma a meghatározott tartományon belül maradjon, lehetséges csomópont-egyesítés és -szétválasztás is. Egy m B-fa kielégíti a következő feltételeket:
- Minden csomópontnak legalább m/2 (kivéve a gyökeret és a leveleket), legfeljebb m leszármazottja lehet
- A gyökérnek legalább 2 leszármazottja van
- Minden levél azonos szinten van, és nem hordoznak információt
- Egy nem-levél csomópont, aminek k leszármazottja van, k-1 kulcsot tárol

Műveletek B-fákon:
- *Keresés*: a bináris fákkal analóg módon működik, az ott bemutatott kétféle algoritmus itt is használható.

- *Beszúrás*: minden művelet a levél elemeken történik. A fán kereséssel meg kell találni az a csomópontot, ahova az új elem beilleszthető. Ha a csomópontnak kevesebb leszármazottja van, mint a maximális megengedett, akkor oda beszúrható az új elem, de figyelembe kell venni, hogy rendezettek maradjanak az elemek. Más esetekben a levelet fel kell bontani két csomópontra. Ki kell jelölni egy középelemet a régi és az új elemek közül. A mediánnál kisebb elemek kerülnek az egyik új, a baloldalon lévő levélre, a nagyobb elemek pedig a másikra, jobboldalra. A medián mint elválasztó érték szerepel. Az elválasztó érték hozzáadódik a szülő elemhez, ami abban a csomópontban okozhat szétválasztást, és így tovább, egészen a gyökér elemig (ha szükséges).
- *Törlés*: kétféle megközelítés jellemző: Kijelölni és törölni az elemet, majd újrastrukturálni a fát; vagy végigmenni a fán egészen a kijelölt csomópontig, de még a csomópont elérése előtt újrastrukturálni a fát, hogy amikor törölve lesz az adott elem, utána már ne kelljen ezzel foglalkozni.
Két speciális eset lehet, amivel még törléskor foglalkozni kellhet:
- egy elem egy csomópontban elválasztó elemként szerepel: a bal oldali leszármazott legnagyobb és a jobboldali leszármazott legkisebb eleme közül valamelyiket ki kell jelölni, mint új elválasztót, és újra kell építeni az alatta lévő struktúrát.
- egy elem törlésével a minimálisan szükségesnél kevesebb leszármazottja lesz egy elemnek: kiegyensúlyozás szükséges
- *Kiegyensúlyozás*: ha egy elem törlésével egy csomópontnak a kötelezően előírtnál kevesebb leszármazottja marad, akkor újra kell építeni a fát olyan módon, hogy mindenhol meglegyen a kötelező minimum. Bizonyos esetekben ez a művelet hiányt képez a szülő elemeknél, így az újraépítés eltarthat egészen a gyökérelemig is.

Modell: egy szoftver által kezelt információ szerkezetének adatfüggetlen, pontos leírása. Az adatokat osztály és példány szerint csoportosítjuk, a tulajdonságokat/értékeket hozzájuk rendeljük. Általánosítunk a közös tulajdonságok kiemelésével.
Metamodell: egy adatszerkezetekre koncentráló modell készítése során használt eszközök és szabályok definíciója.
Az UML négyrétegű metamodell szerkezete:
1. Alkalmazói adatok: a valós világ és a valós adatok szintje, a modellszint példánya
2. Alkalmazói (szoftver) modell: ez a modellszint, a valós világ modellje
3. A modell-leíró formalizmus (pl.: UML) leírása: metamodell, a modell modellje
4. A modellezőeszköz leírásának a leírása: meta-metamodell, a metamodell modellje
Fentről lefelé csökken a bonyolultság, nő az elvontság szintje.
Az UML metamodellje kiterjeszthető ún. profilokkal: ezek a szabványhoz képest valamilyen eltéréseket fogalmaznak meg. Egyik ilyen jellemző profil az OCL (Object Constraint Language), amivel különböző (megszorítási) szabályokat lehet leírni és alkalmazni UML modelleken. Ez egy predikatív nyelv, azaz minden kifejezése kiértékelhető, az eredmény logikai érték. Erősen típusos nyelv, kötelező típusegyeztetéssel. UML rendszerekben használható, így ez is objektumorientált. Megszorító nyelv, azaz példányhalmazhoz kapcsolva megszorítja a halmazt olyan egyedekre, melyekre a kifejezés igazra értékelődik ki.

Modellvezérelt szoftverkészítés: a rendszerek specifikációjának és felépítésének új módja, UML használatával. A teljes fejlesztési ciklust támogatja (elemzés, tervezés, megvalósítás, telepítés, követés, továbbfejlesztés, integráció). Cél a hordozhatóság és az együttműködtethetőség. A modellvezérelt szoftverkészítés alapjait az OMG által készített MDA szabvány fogja össze. **Az MDA alapfogalmai:**
- Formális specifikáció: jól meghatározott forma (nyelvtan/szintaxis), értelmezés (szemantika), elemzési és következtetési szabályok, a szerkezeteket ellenőrző szabályok (formális vagy informális értelmezés megadással)
- Absztrakció: modellek közötti kapcsolat -> a felesleges részletek elhagyása. Finomítás: újabb részletek meghatározása. Absztrakciós kritérium: milyen részletek feleslegesek. Nézet (view): valamilyen kritérium alapján elvégzett modellabsztrakció
- Közelítés, távolítás (zooming in/out) modellművelet: az absztrakció-finomítás kapcsolat mentén történhet
- Platform: technológiai és műszaki részletek összessége, amelyek függetlenek egy szoftver funkcióitól

- Platformfüggő modell (PSM): amely a platform valamelyik elemét már tartalmazza/függ tőle, ellentéte a platform független modell (PIM) (a rendszer szerkezetének és működésének formális meghatározása).

Modellvezérelt szoftver elkészítése akkor lehet szükséges, ha a modellelemeken és a modell adatpéldányain is kívánunk műveleteket végezni, ha a modell futásidőben is módosulhat, vagy ha a modell a szoftver egyik terméke.